

NYU, Tandon School of Engineering

Bridge to CS

Homework #4

Submission Instructions

1. For this assignment, you should turn in 8 `.cpp` files:
 - Questions 1 and 4 require **two separate files** each (parts a and b).
 - Questions 2, 3, 5, and 6 require one file each.
 - Name your files as follows:
 - Q1: `hw4_q1a.cpp`, `hw4_q1b.cpp`
 - Q2: `hw4_q2.cpp`
 - Q3: `hw4_q3.cpp`
 - Q4: `hw4_q4a.cpp`, `hw4_q4b.cpp`
 - Q5: `hw4_q5.cpp`
 - Q6: `hw4_q6.cpp`
 - You should test your code by using a main function. However, do not include any main functions in your final submissions.
 2. Pay special attention to the style of your code. Indent your code correctly, choose meaningful names for your variables, define constants where needed, choose the most appropriate control flow statements, etc.
-

Question 1

Write **two versions** of a function that takes a positive integer `n`, and prints the first `n` even numbers.

Implement the following functions:

- a) `void printEvensWhile(int n)` – uses a `while` loop. Submit as `hw4_q1a.cpp`.
- b) `void printEvensFor(int n)` – uses a `for` loop. Submit as `hw4_q1b.cpp`.

For example, calling `printEvensWhile(3)` or `printEvensFor(3)` should print:

```
2
4
6
```

Question 2

In this question we will use a **simplified** version of the Roman Numerals System to represent positive integers.

The digits in this system are I, V, X, L, C, D and M. Each digit corresponds to a decimal value, as showed in the following table:

Roman digit	I	V	X	L	C	D	M
Decimal value	1	5	10	50	100	500	1000

A number in the *simplified Roman numerals system* is a sequence of Roman digits, which follow these 2 rules:

1. The digits form a monotonically non-increasing sequence. That is the value of each digit is less than or equal to the value of the digit that came before it.

For example, DLXXVI is a monotonically non-increasing sequence of Roman digits, but XIV is not.

2. There is no limit on the number of times that 'M' can appear in the number. 'D', 'L' and 'V' can each appear at most one time in the number. 'C', 'X' and 'I' can each appear at most four times in the number.

For example: IIII, XVII and MMMMMMMDCCLXXXVII are legal numbers in our simplified Roman numeral system, but IIIII, XIV, VVI and CCXLIII are **not**.

Implement the following function:

`void printRoman(int num)` – takes a positive integer and prints its representation in the simplified Roman numerals system.

For example, calling `printRoman(147)` should print:

CXXXVII

Other examples:

- `printRoman(1666)` prints MDCLXVI
- `printRoman(3999)` prints MMMCCCCLXXXVIIII

Question 3

Write a function that takes a positive integer (in decimal representation), and prints its binary (base 2) representation.

Implement the following function:

`void printBinary(int num)` – takes a positive integer and prints its binary representation.

For example, calling `printBinary(76)` should print:

1001100

Other examples:

- `printBinary(1)` prints 1
- `printBinary(15)` prints 1111
- `printBinary(32)` prints 100000

Implementation Requirements:

1. You are supposed to implement the algorithm that converts to base 2. You should not use any `string`, `to_string()`, or special `cout` functionalities to make the conversion.
2. You are not allowed to use arrays.

Question 4

Write two versions of a function that **reads a sequence of positive integers from the user**, calculates their **geometric mean**, and prints the result.

Notes:

1. In mathematics, geometric mean of a dataset $\{a_1, a_2, a_3, \dots, a_n\}$ containing positive numbers, is given by: $\sqrt[n]{a_1 \cdot a_2 \cdot a_3 \cdot \dots \cdot a_n}$.
For example, the geometric mean of 2, 9 and 12 is equal to 6 ($\sqrt[3]{2 \cdot 9 \cdot 12} = 6$).
2. In order to calculate the n-th root of a number, you should call the `pow` function, located in the `cmath` library.

Implement the following functions:

- a) `void geomMeanLength()` – First reads the length of the sequence, then reads that many integers, and prints the geometric mean. Submit as `hw4_q4a.cpp`.

For example, if the user enters 3 followed by 2, 9, 12:

```
3
2
9
12
6
```

- b) `void geomMeanSentinel()` – Keeps reading positive integers until -1 is entered, then prints the geometric mean. Submit as `hw4_q4b.cpp`.

For example, if the user enters 2, 9, 12, -1:

```
2
9
12
-1
6
```

Other examples:

- Geometric mean of $\{1, 2, 3\}$ is approximately 1.8171
- Geometric mean of $\{4, 4, 4\}$ is 4
- Geometric mean of $\{8\}$ is 8

Question 5

Write a function that takes a positive integer `n`, and prints a textual image of an hourglass made of $2n$ lines with asterisks.

Implement the following function:

`void printHourglass(int n)` – prints an hourglass pattern with $2n$ lines.

For example, calling `printHourglass(4)` should print:

```
*****  
*****  
***  
*  
*  
***  
*****  
*****
```

Note: Each line has leading spaces for centering, followed by an odd number of asterisks. No trailing spaces.

Question 6

Write a function that takes a positive integer `n`, and prints all of the numbers from 1 to `n` (inclusive) that have **more even digits than odd digits**.

Implement the following function:

`void printMoreEvenDigits(int n)` – prints all numbers from 1 to `n` that have more even digits than odd digits.

For example, calling `printMoreEvenDigits(30)` should print:

```
2  
4  
6  
8  
20  
22  
24  
26  
28
```

Note: The even digits are 0, 2, 4, 6, 8. The odd digits are 1, 3, 5, 7, 9. A number qualifies only if its count of even digits is **strictly greater than** its count of odd digits (not equal).